

Using OpenCL inside a Docker Container on VOXL2

TABLE OF CONTENTS

- 1 Introduction
- 2 Create a New Directory on VOXL2
- 3 Generate the Required Dependencies using VOXL2 Native OS
- 4 Install Docker on VOXL2
- 5 Create a `Dockerfile` with the following contents in `openc1-docker` directory
- 6 Build the Docker Image
- 7 Create a Test Application
- 8 Build the Test Application
- 9 Run the Test App Outside Docker
- 10 Spin up a Docker Container
- 11 Run the Test App Inside the Docker Container

Introduction

OpenCL can be used within a Docker container on VOXL2 after following the correct setup instructions.

Create a New Directory on VOXL2

```
cd /home/root/  
mkdir -p openc1-docker/deps  
cd openc1-docker/deps
```

BASH

Generate the Required Dependencies using VOXL2 Native OS

```
apt install dpkg-repack  
dpkg-repack qti-adreno  
dpkg-repack libcutils0  
dpkg-repack libsync
```

BASH

```
dpkg-repack qti-libion
dpkg-repack liblog0
```

You should have the following deb packages in `openc1-docker/deps`:

```
ls openc1-docker/deps
libcutils0_0-r1_arm64.deb  liblog0_1.0-r1_arm64.deb  libsync_1.0-r1_arm64.deb  qti-
adreno_1.0-r0_arm64.deb  qti-libion_0-r1_arm64.deb
```

Install Docker on VOXL2

- follow official instructions <https://docs.docker.com/engine/install/ubuntu/> , specifically:

```
curl -fsSL https://get.docker.com -o get-docker.sh
chmod +x get-docker.sh
./get-docker.sh
```

Create a Dockerfile with the following contents in `openc1-docker` directory

```
FROM arm64v8/ubuntu:22.04

# Install necessary dependencies
RUN apt-get update && \
    apt-get install -y \
        cmake \
        build-essential \
        libglib2.0-0

COPY deps /opt/deps

## Create required directory for qti-adreno install
RUN mkdir /usr/include/KHR
RUN dpkg -i /opt/deps/*.deb
```

Build the Docker Image

BASH

```
cd openccl-docker
docker build -t openccl-docker:latest .
```

Create a Test Application

Create a source file `openccl-query.c` with the following contents

```
#include <stdio.h>
#include <stdlib.h>
#include <CL/cl.h>

int main() {

    int i, j;
    char* value;
    size_t valueSize;
    cl_uint platformCount;
    cl_platform_id* platforms;
    cl_uint deviceCount;
    cl_device_id* devices;
    cl_uint maxComputeUnits;

    // get all platforms
    clGetPlatformIDs(0, NULL, &platformCount);

    printf("OpenCL platform count: %d\n", platformCount);

    platforms = (cl_platform_id*) malloc(sizeof(cl_platform_id) * platformCount);
    clGetPlatformIDs(platformCount, platforms, NULL);

    for (i = 0; i < platformCount; i++) {

        // get all devices
        clGetDeviceIDs(platforms[i], CL_DEVICE_TYPE_ALL, 0, NULL, &deviceCount);
        printf("OpenCL device count: %d\n", deviceCount);

        devices = (cl_device_id*) malloc(sizeof(cl_device_id) * deviceCount);
        clGetDeviceIDs(platforms[i], CL_DEVICE_TYPE_ALL, deviceCount, devices, NULL);
```

```
// for each device print critical attributes
for (j = 0; j < deviceCount; j++) {

    // print device name
    clGetDeviceInfo(devices[j], CL_DEVICE_NAME, 0, NULL, &valueSize);
    value = (char*) malloc(valueSize);
    clGetDeviceInfo(devices[j], CL_DEVICE_NAME, valueSize, value, NULL);
    printf("%d. Device: %s\n", j+1, value);
    free(value);

    // print hardware device version
    clGetDeviceInfo(devices[j], CL_DEVICE_VERSION, 0, NULL, &valueSize);
    value = (char*) malloc(valueSize);
    clGetDeviceInfo(devices[j], CL_DEVICE_VERSION, valueSize, value, NULL);
    printf(" %d.%d Hardware version: %s\n", j+1, 1, value);
    free(value);

    // print software driver version
    clGetDeviceInfo(devices[j], CL_DRIVER_VERSION, 0, NULL, &valueSize);
    value = (char*) malloc(valueSize);
    clGetDeviceInfo(devices[j], CL_DRIVER_VERSION, valueSize, value, NULL);
    printf(" %d.%d Software version: %s\n", j+1, 2, value);
    free(value);

    // print c version supported by compiler for device
    clGetDeviceInfo(devices[j], CL_DEVICE_OPENCL_C_VERSION, 0, NULL, &valueSize);
    value = (char*) malloc(valueSize);
    clGetDeviceInfo(devices[j], CL_DEVICE_OPENCL_C_VERSION, valueSize, value,
NULL);

    printf(" %d.%d OpenCL C version: %s\n", j+1, 3, value);
    free(value);

    // print parallel compute units
    clGetDeviceInfo(devices[j], CL_DEVICE_MAX_COMPUTE_UNITS,
        sizeof(maxComputeUnits), &maxComputeUnits, NULL);
    printf(" %d.%d Parallel compute units: %d\n", j+1, 4, maxComputeUnits);

}
```

```
        free(devices);  
    }  
  
    free(platforms);  
    return 0;  
}
```

Build the Test Application

```
gcc -O2 -Wall -o openc1-query openc1-query.c -lOpenCL
```

Run the Test App Outside Docker

```
voxl2:~/openc1-docker$ ./openc1-query  
OpenCL platform count: 1  
OpenCL device count: 1  
1. Device: QUALCOMM Adreno(TM)  
  1.1 Hardware version: OpenCL 2.0 Adreno(TM) 650  
  1.2 Software version: OpenCL 2.0 QUALCOMM build: commit # changeid # Date: 11/10/21 Wed  
Local Branch: Remote Branch: Compiler E031.37.12.01  
  1.3 OpenCL C version: OpenCL C 2.0 Adreno(TM) 650  
  1.4 Parallel compute units: 3
```

Spin up a Docker Container

- `--privileged` flag maps all the linux devices to the container
- if this is a concern, the `--privileged` flag can be replaced with `--device=/dev/kgsl-3d0 --device=/dev/ion`

```
docker run -it --rm --privileged -v `pwd`: /opt/code -w /opt/code openc1-docker:latest  
bash
```

BASH

Run the Test App Inside the Docker Container

```
root@f23c81c4a254:/opt/code# ./openc1-query
```

```
OpenCL platform count: 1
```

```
OpenCL device count: 1
```

```
1. Device: QUALCOMM Adreno(TM)
```

```
1.1 Hardware version: OpenCL 2.0 Adreno(TM) 650
```

```
1.2 Software version: OpenCL 2.0 QUALCOMM build: commit # changeid # Date: 11/10/21 Wed
```

```
Local Branch: Remote Branch: Compiler E031.37.12.01
```

```
1.3 OpenCL C version: OpenCL C 2.0 Adreno(TM) 650
```

```
1.4 Parallel compute units: 3
```

Copyright © 2019-2026 ModalAI Inc.

Page last modified: Sep 3 2026 at 01:24 PM.